CS 3110/3990: Data Privacy

In-Class Exercise, Week of 9/22/2025

```
# Load the data and libraries
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

def laplace_mech(v, sensitivity, epsilon):
    return v + np.random.laplace(loc=0, scale=sensitivity / epsilon)

def pct_error(orig, priv):
    return np.abs(orig - priv)/orig * 100.0

adult =
pd.read_csv('https://github.com/jnear/cs3110-data-privacy/raw/main/
homework/adult_with_pii.csv')
```

Question 1

For various values of b, write code to print out the percent error of summing the ages in the `adult` dataset, clipped to each value of b.

```
bs = range(1, 100, 10)
real_sum = adult['Age'].sum()

for b in bs:
    clipped_sum = adult['Age'].clip(upper=b).sum()
    print(b, pct_error(real_sum, clipped_sum))
```

```
1 97.4080940444511
11 71.48903448896206
21 46.000380495392264
31 25.509031989473492
41 11.7959939725709
51 4.329687317165198
61 1.1587597123836921
71 0.2341877497996031
81 0.03860675005194001
91 0.0
```

What value of b is the best?

91 is the best b value

Question 2

For various values of b, print the result of a *differentially private* sum of ages, clipped to each value of b. Use $\epsilon = 0.1$.

```
bs = range(1, 100, 10)
real_sum = adult['Age'].sum()

for b in bs:
    clipped_sum = adult['Age'].clip(upper=b).sum()
    dp = laplace_mech(clipped_sum, sensitivity=b, epsilon=0.1)
    print(b, pct_error(real_sum, dp))
```

```
1 97.40708710457582
11 71.4884928272845
21 46.00341570350942
31 25.546908315666517
41 11.769695509536131
51 4.359996385070372
61 1.1283169033991094
71 0.19948528659777895
81 0.04369989547575415
91 0.008742494403835396
```

Which value of b is the best now? Does it differ?

91 is still seems to be the best b value

Question 3

Write an algorithm to *automatically pick the clipping parameter* for the age column. Your algorithm should satisfy differential privacy.

```
def pick_b(epsilon):
    bs = range(1,200,10)
    last_result = 0
    epsilon_i = epsilon /len(bs)

    for b in bs:
        clipped_sum = adult['Age'].clip(upper=b).sum()
        result = laplace_mech(clipped_sum, sensitivity=b,
epsilon=epsilon_i)
        # heuristic: if result < last_result then we found a decrease
and should stop
        # this decrease means we passed the plateau
        if result < last_result:
            return b
```

```
        else:
            last_result = result


#total privacy cost = len(bs) * epsilin_i = len(bs)*(epsilon/len(bs))
= epsilon

pick_b(1.0)
```

```
81
```

```
# TEST CASE for question 3

many_trials = np.mean([pick_b(1.0) for _ in range(100)])
assert many_trials > 70
assert many_trials < 500
```

Question 4

What is the privacy cost of your algorithm, and why?

1. Sensetivity: the laplace mechanism is applied ti a clipped sum, for which the sensetivity is equal to clipping parameter
2. Composition:
    a. I cakk the laplace nechanism once for each iteration ofn the loop with privacy paarmeter epsilon_i
    b. the loop runs len(bs) times
    c. the total privacy cost is len(bs) * epsilon_i by sequential composition
    d. len(bs) * epsilon_i = epsilon by definitionnof epsilon_i
3. Postprocessing:
    a. WE return b (which does not depend on the data, so it satisfies the diferential privacy)
    b. We decide "which" bto return by looking at differentinaly private results
    c. In the worst case, we examine all of the DP results =, for a total privacy cost of epsilon

Question 5

Write code to generate a *histogram* of education numbers in the `adult` dataset.

```
def education_hist():
    # return adult.groupby('Education').size()
    return adult['Education'].value_counts()

education_hist()
```

```
Education
HS-grad         10501
```

```
Some-college     7291
Bachelors        5355
Masters          1723
Assoc-voc        1382
11th             1175
Assoc-acdm       1067
10th              933
7th-8th           646
Prof-school       576
9th               514
12th              433
Doctorate         413
5th-6th           333
1st-4th           168
Preschool          51
Name: count, dtype: int64
```

```
# TEST CASE for question 5
h = education_hist()
assert h['HS-grad'] == 10501
assert h['12th'] == 433
assert h['Doctorate'] == 413
```

Question 6

Write code to release a *differentially private histogram* of education numbers.

```
def dp_education_hist(epsilon):
    hist = education_hist()
    #each element if the histogram is a count, so it has sensetivity 1
    noisy_hist = hist.apply(lambda x: laplace_mech(x, sensitivity=1,
epsilon=epsilon))
    return noisy_hist


dp_education_hist(1.0)
```

```
Education
HS-grad         10501.031032
Some-college     7291.415916
Bachelors        5353.956693
Masters          1723.216571
Assoc-voc        1381.156546
11th             1172.630561
Assoc-acdm       1067.337355
10th              932.927547
7th-8th           645.758541
Prof-school       576.287395
9th               513.880669
```

```
12th                432.089413
Doctorate           411.232877
5th-6th             334.456798
1st-4th             167.346250
Preschool            51.518969
Name: count, dtype: float64
```

```
# TEST CASE for question 6
h = dp_education_hist(1.0)
assert abs(h['HS-grad'] - 10501) < 100
assert abs(h['Doctorate'] - 413) < 100
```

Question 7

What is the total privacy cost of `dp_education_hist`, and why?

1. sensetivity
   - each element of a histogram is a count so its sensetivity is 1
2. composition
   - we run the laplace nechanism len(hist) times
   - for a total privacy cost of len(hist) * (epsilon/len(hist)) = epsilon by sequentioal cmposition
3. post processing -we just return the noisy histogram at the end

Question 8

Write code to generate a differentially private *contingency table* for the Education and Sex columns of the `adult` dataset.

```
def dp_crosstab_education_sex(epsilon):
    hist = adult[['Education', 'Sex']].value_counts()
    hist = adult.groupby(['Education', 'Sex']).size()
    hist = pd.crosstab(adult['Education'], adult['Sex'])

    return hist.map(lambda x: laplace_mech(x, sensitivity=1,
epsilon=epsilon))

dp_crosstab_education_sex(1.0)
```

```
Sex                  Female         Male
Education
10th             294.077706   638.484204
11th             432.624396   745.565385
12th             144.341694   285.458372
1st-4th           44.088363   121.629121
5th-6th           84.766281   247.725499
7th-8th          160.826699   483.279045
9th              144.228370   372.851408
```

```
Assoc-acdm     421.015183   647.386975
Assoc-voc      500.345489   882.476221
Bachelors     1619.028860  3737.147596
Doctorate       85.594190   333.268350
HS-grad       3389.338371  7111.952374
Masters        536.234883  1189.562278
Preschool       15.459123    36.843268
Prof-school     93.692789   481.853910
Some-college  2806.209599  4485.062078
```

```
# TEST CASE for question 8
ct = dp_crosstab_education_sex(1.0)
assert abs(ct['Female']['10th'] - 295) < 100
assert abs(ct['Male']['10th'] - 638) < 100
assert abs(ct['Female']['Bachelors'] - 1619) < 100

assert abs(ct['Female']['10th'] - 295) > 0
assert abs(ct['Male']['10th'] - 638) > 0
assert abs(ct['Female']['Bachelors'] - 1619) > 0
```

Question 9

Does parallel composition apply for the contingency table in question 1? Why or why not?

Yes, parallel composition still aplies since paralel substence we are countig is still disjoint. It is regardless of the dementionality of the histogram

Question 10

Does the number of columns used in constructing the contingency table matter for privacy cost? Does it matter for accuracy?

Privacy: no the number of colums matter, the bins of the histgoram will always be disjoint. so i cam alwasy use paralel co posiion Accuracy: yes the number of ciollumns matters a lot fot relative errot/ utility - as the dimensiomnality increases the number if the people in each histogrsm bit decreases exponentialy so the signak to noise ratioof each noisy hisyogeam gets worse so bad that peope almost never do more than 3 dimensions

Question 11

Implement the Gaussian mechanism for (ϵ, δ)-differential privacy.

```
# YOUR CODE HERE
raise NotImplementedError()
```

```
---------------------------------------------------------------------------
NotImplementedError                       Traceback (most recent call last)
```

```
Cell In[60], line 2
      1 # YOUR CODE HERE
----> 2 raise NotImplementedError()

NotImplementedError:
```

```
# TEST CASE

results = [gaussian_mech(len(adult[adult['Age'] > 50]), 1, 1.0, 10e-5)
for _ in range(100)]
errors = [pct_error(len(adult[adult['Age'] > 50]), r) for r in
results]
print('mean error:', np.mean(errors))

assert np.mean(errors) > 0
assert np.mean(errors) < 2
```

Question 12

How do the Laplace and Gaussian mechanisms compare in terms of relative error on the query "how many individuals are over 50 years old" with $\epsilon = 1$ and $\delta = 10^{-5}$?

```
true_answer = len(adult[adult['Age'] > 50])

laplace_answers = [laplace_mech(true_answer, 1, 1) for _ in
range(200)]
gaussian_answers = [gaussian_mech(true_answer, 1, 1, 10e-5) for _ in
range(200)]

laplace_error = [pct_error(true_answer, a) for a in laplace_answers]
gaussian_error = [pct_error(true_answer, a) for a in gaussian_answers]

_, bins, _ = plt.hist(gaussian_error, bins=20, label='Gaussian')
plt.hist(laplace_error, bins=bins, label='Laplace', alpha=0.5)
plt.legend();
```

YOUR ANSWER HERE

Not a Question - Just for reference

Reference

```
epsilon = 1
sensitivity = 1
delta = 1e-5
sigma_squared = 2 * sensitivity**2 * np.log(1.25 / delta) /
(epsilon**2)
sigma = np.sqrt(sigma_squared)
```

```python
def gauss_pdf(x):
    return 1/(sigma*np.sqrt(2*np.pi)) * np.exp(-(1/2)*(x/sigma)**2)

xs = np.linspace(-50, 50, 200)
ys1 = [gauss_pdf(x) for x in xs]
ys2 = [gauss_pdf(x+1) for x in xs]

plt.plot(xs,ys1)
plt.plot(xs,ys2)

# ratio < e^epsilon should hold
print('e^epsilon =', np.exp(epsilon))
ratios = [(x, y1 / y2) for x, y1, y2 in zip(xs, ys1, ys2)]
#ratios

def laplace_pdf(x):
    return (1/2)*epsilon * np.exp(-np.abs(x)*epsilon)

xs = np.linspace(-50, 50, 200)
ys1 = [laplace_pdf(x) for x in xs]
ys2 = [laplace_pdf(x+1) for x in xs]

plt.plot(xs,ys1)
plt.plot(xs,ys2)

# ratio < e^epsilon should hold
print('e^epsilon =', np.exp(epsilon))
ratios = [(x, y1 / y2) for x, y1, y2 in zip(xs, ys1, ys2)]
#ratios
```